ACS-2947-002/003

Lab 5

Due by Tuesday, March 4, 11:59 PM

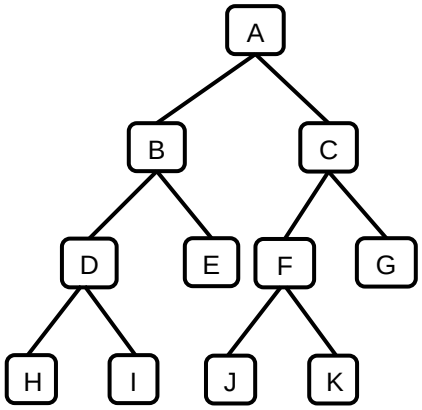
Instructions

- The goal of this lab is to understand the implementation of the Tree ADT and learn to use the different methods of the ADT (without an Iterator).
- Submit your `.java` files (together in a `Lab5.zip` file) via Nexus.
- Include your name and student number as a comment in every file.

Task

1. Create a class called `LinkedBinaryTree` that implements the `BinaryTree` (which extends `Tree`) and `Position` interfaces. Use the `AbstractBinarytree` and `AbstractTree` classes in your notes/textbook to provide a base for your linked binary tree. Work with your notes from class. Note that:
   - The provided `Tree` interface is a modified version that does **NOT** include `iterator()` and `positions()`. We will add the tree traversal/iterator in Assignment 3.
   - The `attach()` method is **NOT** required.
   - `toString()` for `LinkedBinaryTree` is **NOT** required for this lab (we will implement Iterators in Assignment 3).

2. In your `Lab5_Driver` class, create a static tree instance and build a simple tree of names that is similar in structure (but not content) to the following tree. That is, the tree contains 11 names. **You must use valid names of your choice instead of the letters.**



3. Include the following **static** methods in your `Lab5_Driver`:
   - `allDescendants` that takes a node/position as parameter and **recursively** displays all the descendants of that position.
     - Note how the `children()` method returns an `Iterable`: you should use this to traverse through the list of children of a node and recursively display their children.

For example, using the node named *B* above, the method displays:

```
D, H, I, E
```

- Illustrate this method by displaying all the descendants of the root node.
- Illustrate this method by displaying all the descendants of the left child of the root node. *You must use the left() method in this.*

- `pathToroot` that displays the path from any given node/position to the root of the tree.
  - Think of this as a position being traversed through the path to get from the position to the root and consider how we have traversed through a linked list from the first to last.

  For example, for the given tree, the following shows the path from the node *I* upwards to the root:

  ```
  The path from I to root(A): I, D, B, A
  ```

  Illustrate this method by displaying the path from the **second leftmost node at the deepest level** e.g., node with element I above.

4. In your `Lab5_Driver` , include the code to do the following:
   - Display the height of the tree.
   - Display the depth of the right child of the root node. *You must use the right() method in this.*
   - Remove the node at the leftmost position of the deepest level. E.g., node H above
   - Display the final size of the tree.

5. Display the output as shown below.

Sample Output

```
The descendants of A: ...
The descendants of B: ...
The path from I to root(A): ...
Tree Height: ...
Depth of right child of the root (C): ...
Removed node: I
Final tree size: ...
```

Submission

Submit your **`Lab5.zip`** file that includes all the following files (`Position.java`, `Tree.java`, `AbstractTree.java`, `BinaryTree.java`, `AbstractBinaryTree.java`, `LinkedBinaryTree.java`, `Lab5_Driver.java`) and any other files you use via **Nexus**.

EXTRA WORK: Do not submit

1. Using your `LinkedPositionalList` implementation from Lab 4 (or the sample solution for lab 4), add a position and element iterator.
   a) Have the `PositionalList` interface extend `Iterable`. Add the empty methods `iterator()` and `positions()` that return `Iterator<E>` and `Iterable<Position<E>>`, respectively.
   b) Add the nested classes and methods from your notes/text.
      - `PositionIterator, PositionIterable, and positions()`
      - `ElementIterator and iterator()`

      Note that `PositionalList`, `LinkedPositionalList` and `ExtraWork_Driver` require `java.util.Iterator`

2. The provided `ExtraWork_Driver` includes a populated positional list of integers. Complete the driver with the following requirements. Display the list after each step.
   a) Declare and initialize an *element* iterator. Iterate through the list and remove the positions where elements are a 1 or 2.
   b) Using the *position* enhanced for, replace every 4 with 3.
   c) With a new instance of element iterator, remove all consecutive duplicates.
      i.e., for list 1 2 2 3 3 3 4, the result would be 1 2 3 4.
   d) Using the *element* enhanced **for** statement, calculate the sum of integers.

3. Create a class called RecursionDemo that contains the following generic static methods:
   a) `clearStack()` that ***recursively*** empties a stack
      - Create and clear a stack, showing the before and after results in your output.

      Note that you may use your implementation Stack from our ADT or `java.util.Stack`
   b) `numDigits()` method that uses recursion to determine the number of digits in a positive integer (recall that the number of digits is $\lfloor \log_{10} n \rfloor + 1$).
      - Use the `Scanner` object to read the integer input from the keyboard and display the result in your output

4. Reimplement our `ArrayQueue` using dynamic arrays and the provided `Queue` interface. Note that the queue interface extends `Iterable`. A partial implementation of the new `ArrayQueue` class is provided for you to work with. You will need to add the following:
   - Private `resize` method
   - `enqueue` and `toString` methods
   - `next` method in the `ArrayIterator` inner class
   - `clear` method that removes all elements from the underlying array in O(1) run time

   Create a driver class to test that your implementation works.

Note:

- The circular implementation of the `ArrayQueue` class must remain so that the `enqueue`/`dequeue` operations are performed in O(1) run time.
- The initial capacity of the underlying array must be 4.
- Make sure to consider the circular structure of the `ArrayQueue` in your implementation of the `resize` and `toString` methods. For the resize method, the front may reset to 0 after copying the elements, i.e. the front element is copied to index 0.
- The `remove` method of the `ArrayIterator` class is not supported for this class.
- Do not iterate over the `ArrayQueue` to implement a `clear` method, think about how this may be achieved in O(1) run time.
- In your implementation, include the following line before the call to the `resize` method. `System.out.println("...Resizing array...");`